How the Code Works — every file, step by step

This guide walks through **all the code in `firmware/`, instruction by instruction**, in simple words for a young maker. Read it from top to bottom. Every time you see a piece of code, the words right after explain what it does.

0. A few Rust words first

Rust is read from top to bottom. Here are the words we will meet, in plain language:

Rust word	Simple meaning
`let`	Make a box with a name and put a value inside
`mut`	"Changeable": the value in the box can change
`fn`	A function: a small machine with a name that does a job
`pub`	Public: other files are allowed to use it
`const`	A fixed number that never changes
`enum`	A list of choices (only one can be true)
`struct`	A box that holds several values together
`impl`	"The abilities of" a type (its functions)
`trait`	A promise: "I can do this job"
`match`	Choose what to do by comparing
`Option`	Maybe a value: `Some(x)` or nothing `None`
`Result`	Works `Ok(x)` or fails `Err(e)`
`?`	"If this failed, stop and hand back the error"
`&mut`	Borrow something so you are allowed to change it

`#[...]`	A note that tells the compiler something special
`<T>`	A placeholder for a type, filled in later

1. The map of the project

```
firmware/
 Cargo.toml              settings for the whole project
 build.rs                a helper that runs before the build
 memory.x                a map of the Pico's memory
 .cargo/config.toml      settings for how to build and run
 src/
   main.rs               THE BODY: sets up hardware, runs the loop
   lib/
     mod.rs              THE BRAIN: list of brain modules
     state_machine.rs    decides the mood
     sensor.rs           measures distance (the eyes)
     servo.rs            turns the head (the neck)
     audio.rs            plays the song (the voice)
```

`src/main.rs` is the **body**: it touches the real hardware. `src/lib/` is the **brain**: pure thinking, easy to test. This split is on purpose, so the clever logic can be tested on a normal computer.

2. The project settings (not instructions, just settings)

2.1 `Cargo.toml`

This file tells Rust the name of the project and which helper libraries ("crates") it may use.

```
[package]
name = "scary-monster-firmware"
version = "0.1.0"
edition = "2021"
rust-version = "1.75"
```

- `[package]` starts the description of our project.
- `name` is the project name.
- `version` is its version number (`0.1.0` means "first try").
- `edition = "2021"` picks the modern Rust rules.
- `rust-version = "1.75"` says we need at least Rust 1.75.

```
[lib]
name = "scary_monster_lib"
path = "src/lib/mod.rs"
```

- This says: "the library part of this project lives in `src/lib/`".
- `name` is how other code calls it: `scary_monster_lib`.

```
[dependencies]
embedded-hal = "1.0"
embedded-io = "0.6"
```

- `dependencies` are the helper libraries we borrow.
- `embedded-hal` gives us the standard "shapes" of hardware (pins, PWM...).
- `embedded-io` gives us the standard "shape" for sending bytes (for the UART).

```
[target.'cfg(target_arch = "arm")'.dependencies]
cortex-m = "0.7"
cortex-m-rt = "0.7"
defmt = "0.3"
defmt-rtt = "0.4"
panic-halt = "0.2"
rp-pico = "0.9"
```

- These helpers are only for the **Pico** (an ARM chip), not for a normal computer. That is why they are under the "arm only" section.
- `cortex-m-rt` starts our program on the chip.
- `panic-halt` says "if something goes very wrong, just stop" (safe for a prop).
- `defmt` + `defmt-rtt` let us print little log messages.
- `rp-pico` knows all the Pico's parts.

```
[profile.release]
opt-level = "s"
lto = true
debug = 2
```

- These are the settings for the "release" (final) build.
- `opt-level = "s"` makes the program **small** (good for a tiny chip).
- `lto = true` lets the compiler clean up across files.
- `debug = 2` keeps a little information so logs show line numbers.

2.2 `build.rs` — the little helper that runs first

```
use std::env;
use std::fs::File;
use std::io::Write;
use std::path::PathBuf;

fn main() {
   let out = PathBuf::from(env::var_os("OUT_DIR").unwrap());
   File::create(out.join("memory.x"))
       .unwrap()
       .write_all(include_bytes!("memory.x"))
       .unwrap();
   println!("cargo:rustc-link-search={}", out.display());
   println!("cargo:rerun-if-changed=memory.x");
}
```

- `use ...` lines bring in tools (environment, file, writing, paths).
- `fn main()` is this helper's job.
- `let out = ...` finds the folder where the build puts its temporary files.
- `File::create(out.join("memory.x"))` makes a file called `memory.x` there.
- `.write_all(include_bytes!("memory.x"))` copies our `memory.x` into it.
- `println!("cargo:rustc-link-search={}", ...)` tells the build "look here for the memory map".
- `println!("cargo:rerun-if-changed=memory.x")` says "if `memory.x` changes, run me again".

This helper does **not** run on the Pico. It runs on your computer, once, before building.

2.3 `memory.x` — the map of the memory

```
MEMORY
{
   BOOT2 : ORIGIN = 0x10000000, LENGTH = 0x100
   FLASH : ORIGIN = 0x10000100, LENGTH = 2048K
   RAM   : ORIGIN = 0x20000000, LENGTH = 264K
}
```

- This tells the linker where things live on the chip.
- `BOOT2` is a tiny 256-byte area (`0x100`) at the very start of flash that helps the chip start.
- `FLASH` is the big program storage: 2048K (2 megabytes) starting right after the boot area.
- `RAM` is the fast working memory: 264K starting at `0x20000000`.

2.4 `.cargo/config.toml` — how to build and run

```
[build]
target = "thumbv6m-none-eabi"
```

- The Pico's brain chip is an ARM Cortex-M0+. Its Rust name is `thumbv6m-none-eabi`. This line says "build for that chip by default".

```
[target.thumbv6m-none-eabi]
runner = "probe-rs run --chip RP2040"
rustflags = [
   "-C",
   "link-arg=-Tlink.x",
   "-C",
   "link-arg=-Tdefmt.x",
]
```

- `runner` is the program used to send the code to the Pico (`probe-rs`).
- `rustflags` are extra instructions for the linker:
- `-Tlink.x` uses the memory map,
- `-Tdefmt.x` sets up the log system.

```
[env]
DEFMT_LOG = "info"
```

- `DEFMT_LOG = "info"` says "show me the normal log messages".

3. The brain: `src/lib/mod.rs`

```
#![cfg_attr(not(test), no_std)]

pub mod audio;
pub mod sensor;
pub mod servo;
pub mod state_machine;
```

- `#![cfg_attr(not(test), no_std)]` means: "when we are **not** running tests, do not use the big standard library (`no_std`)". On a tiny chip there is no operating system, so we must be small. When we run tests on the computer, we may use the normal library.
- `pub mod audio;` etc. say "the brain has these four parts, and other files may use them". Each `mod` points to a file with the same name (`audio.rs`, `sensor.rs`, ...).

4. `src/lib/state_machine.rs` — deciding the mood

4.1 The explanation at the top

```
//! Behavior finite state machine.
//!
//! The prop has exactly three states. The next state is a pure function of the
//! nearest distance reported by the two side sensors, which keeps the logic
//! small and easy to test.
```

- Lines starting with `//!` are notes that describe the whole file. They are not instructions; they are for humans.

4.2 The three important numbers

```
pub const FOLLOW_DISTANCE_CM: u16 = 100;
pub const ACTION_DISTANCE_CM: u16 = 30;
pub const TURN_DEADBAND_CM: u16 = 10;
```

- `const` means a fixed number that never changes.
- `FOLLOW_DISTANCE_CM` = 100: if someone is closer than 100 cm, follow them.
- `ACTION_DISTANCE_CM` = 30: if someone is closer than 30 cm, play the song.
- `TURN_DEADBAND_CM` = 10: ignore differences smaller than 10 cm, so the head does not wobble.
- `u16` means "a whole number from 0 to 65535".

4.3 The three moods

```
#[derive(Debug, Clone, Copy, PartialEq, Eq, Default)]
pub enum State {
   /// Waiting; nothing is within range.
   #[default]
   Idle,
   /// A person is within `FOLLOW_DISTANCE_CM`; follow their movement.
   Follow,
   /// A person is within `ACTION_DISTANCE_CM`; play the scary song.
   Action,
}
```

- `enum` is a list of choices. Here the monster can only be in one of three moods.
- `Idle` = waiting, `Follow` = following, `Action` = playing the song.
- `#[derive(...)]` is a shortcut that asks Rust to create useful abilities for us: `Debug` (print it), `Clone`/`Copy` (can be duplicated), `PartialEq`/`Eq` (can be compared with `==`), and `Default`.
- `#[default]` marks `Idle` as the starting mood.

4.4 Which way to look

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Direction {
   Left,
   Center,
   Right,
}
```

- Another list of choices: the head can point `Left`, `Center`, or `Right`.

4.5 The reading box

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Reading {
   pub left_cm: Option<u16>,
   pub right_cm: Option<u16>,
}
```

- A `struct` is a box that holds several values together.
- `left_cm` is the left eye's distance, `right_cm` the right eye's.
- `Option<u16>` means "maybe a number, maybe nothing". Nothing (`None`) means the sensor did not see anything. A number arrives as `Some(x)`.

4.6 `nearest_cm()` — the closest person

```
impl Reading {
   /// The smallest valid distance across both sensors.
   pub fn nearest_cm(&self) -> Option<u16> {
       match (self.left_cm, self.right_cm) {
           (Some(left), Some(right)) => Some(left.min(right)),
           (Some(left), None) => Some(left),
           (None, Some(right)) => Some(right),
           (None, None) => None,
       }
   }
```

- `impl Reading` means "these are the abilities of a `Reading`".
- `pub fn nearest_cm(&self)` is a function that looks at itself (`&self`) and answers the smallest distance.
- `match (self.left_cm, self.right_cm)` looks at both eyes at once:
- both see something -> take the smaller (`left.min(right)`),
- only the left sees -> use the left,
- only the right sees -> use the right,
- neither sees -> answer `None`.

4.7 `direction()` — left, centre, or right

```
    /// Where the head should point, based on the closest side.
   pub fn direction(&self) -> Direction {
       match (self.left_cm, self.right_cm) {
           (Some(left), Some(right)) => {
               if left + TURN_DEADBAND_CM < right {
                   Direction::Left
               } else if right + TURN_DEADBAND_CM < left {
                   Direction::Right
               } else {
                   Direction::Center
               }
           }
           (Some(_), None) => Direction::Left,
           (None, Some(_)) => Direction::Right,
           (None, None) => Direction::Center,
       }
   }
}
```

- If both eyes see: compare them.
- If the left is clearly closer (by more than the deadband), look `Left`.
- Else if the right is clearly closer, look `Right`.
- Otherwise look `Center`.
- The _ means "I do not care about the exact number here".
- If only the left eye sees, the person must be on the left, so `Left`.
- If only the right eye sees, `Right`.
- If nobody is seen, look `Center`.

4.8 The mood machine itself

```
#[derive(Debug, Default)]
pub struct StateMachine {
   state: State,
}
```

- A `StateMachine` is a box that remembers the **current** mood.
- `#[derive(Default)]` gives a starting value (which is `Idle`).

```
impl StateMachine {
   /// Creates a state machine in the `Idle` state.
   pub fn new() -> Self {
```

```
        Self::default()
    }

    /// The current state.
    pub fn state(&self) -> State {
        self.state
    }
```

- `new()` builds a fresh machine, starting in `Idle`.
- `state()` answers the current mood (we need this to know when the mood **changes**).

```
    /// Advances the machine with a new sensor reading and returns the state.
    pub fn update(&mut self, reading: Reading) -> State {
        self.state = match reading.nearest_cm() {
            Some(distance) if distance <= ACTION_DISTANCE_CM => State::Action,
            Some(distance) if distance <= FOLLOW_DISTANCE_CM => State::Follow,
            _ => State::Idle,
        };
        self.state
    }
}
```

- `update(&mut self, ...)` is allowed to **change** the machine (`&mut`).
- It looks at the nearest distance and picks a mood:
- 30 cm or less -> `Action`,
- 100 cm or less -> `Follow`,
- anything else (including "nothing seen") -> `Idle`.
- The chosen mood is stored in `self.state` and also returned.

4.9 The tests

```
#[cfg(test)]
mod tests {
    use super::*;
    ...
}
```

- `#[cfg(test)]` means "only build this part when running tests".
- `use super::*;` brings in everything from the file above, so tests can use it.

```
    fn reading(left_cm: Option<u16>, right_cm: Option<u16>) -> Reading {
        Reading { left_cm, right_cm }
```

```
    }
```

- A tiny helper to make a `Reading` quickly in tests.

```
     #[test]
    fn idle_when_nothing_is_detected() {
        let mut machine = StateMachine::new();
        assert_eq!(machine.update(reading(None, None)), State::Idle);
    }
```

- `#[test]` marks a test. `assert_eq!` checks that two things are equal; if not, the test fails.
- This test says: with no readings, the mood must be `Idle`.

The other tests work the same way and check: far away -> `Idle`, 80 cm -> `Follow`, 20 cm -> `Action`, stepping back goes to `Follow`, leaving goes to `Idle`, and that `direction()` picks the correct side.

5. `src/lib/sensor.rs` — the eyes

5.1 The clock promise

```
use embedded_hal::digital::{InputPin, OutputPin};

pub trait MicrosClock {
    fn now_micros(&mut self) -> u32;
}
```

- `use ...` brings in the standard shapes of input and output pins.
- `trait MicrosClock` is a promise: "I can tell you how many microseconds have passed". A `u32` is a whole number from 0 to about 4 billion.
- We use a promise instead of a real clock so we can swap in a fake clock during tests.

5.2 The sensor promise and errors

```
pub trait DistanceSensor {
    type Error;

    fn distance_cm(&mut self) -> Result<u16, Self::Error>;
}
```

- Any "distance sensor" must be able to answer a distance in cm, or give an error. `Result<u16, Self::Error>` means "a number, or a failure".

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum SensorError {
   Timeout,
   Pin,
}
```

- Two ways a sensor can fail: `Timeout` (no echo came back) or `Pin` (a pin misbehaved).

5.3 Numbers for the HC-SR04

```
pub const MAX_RANGE_CM: u16 = 400;

const TRIGGER_PULSE_US: u32 = 10;
const ECHO_TIMEOUT_US: u32 = 30_000;
const US_PER_CM: u32 = 58;
```

- `MAX_RANGE_CM` = 400: readings farther than this are unreliable.
- `TRIGGER_PULSE_US` = 10: we send a 10-microsecond "ping".
- `ECHO_TIMEOUT_US` = 30_000: wait at most 30 milliseconds for an echo.
- `US_PER_CM` = 58: sound needs about 58 microseconds per centimetre (there and back).

5.4 The sensor box

```
pub struct HcSr04<Trig, Echo> {
   trig: Trig,
   echo: Echo,
}
```

- `HcSr04<Trig, Echo>` is a generic box: it can hold **any** trigger pin and **any** echo pin. The `<Trig, Echo>` are placeholders filled in when we build it.

```
impl<Trig, Echo> HcSr04<Trig, Echo>
where
   Trig: OutputPin,
   Echo: InputPin,
{
   pub fn new(trig: Trig, echo: Echo) -> Self {
       Self { trig, echo }
   }
```

- `where Trig: OutputPin, Echo: InputPin` says the trigger must be an **output** pin and the echo an **input** pin.
- `new()` simply stores the two pins in the box.

5.5 `distance_cm()` — one measurement

```
    pub fn distance_cm<Clock: MicrosClock>(
        &mut self,
        clock: &mut Clock,
    ) -> Result<u16, SensorError> {
        self.trig.set_low().map_err(|_| SensorError::Pin)?;
        self.delay_us(clock, 2);
        self.trig.set_high().map_err(|_| SensorError::Pin)?;
        self.delay_us(clock, TRIGGER_PULSE_US);
        self.trig.set_low().map_err(|_| SensorError::Pin)?;
```

- `map_err(|_| SensorError::Pin)?` means: "if setting the pin failed, turn it into our `Pin` error and stop".
- Make TRIG low, wait 2 microseconds, then high for 10 microseconds, then low again. That short high pulse is the "ping!".

```
        let start = self.wait_for_echo(clock, true)?;
        let end = self.wait_for_echo(clock, false)?;
        let elapsed_us = end.wrapping_sub(start);

        Ok((elapsed_us / US_PER_CM).min(u32::from(MAX_RANGE_CM)) as u16)
    }
```

- Wait until ECHO goes **high** (`true`) and remember the time (`start`).
- Wait until ECHO goes **low** (`false`) and remember the time (`end`).
- `end.wrapping_sub(start)` is the time ECHO stayed high.
- Divide by 58 to get centimetres, cap it at 400, and return it.
- `as u16` turns the number into the small kind we use elsewhere.

5.6 `delay_us()` — a tiny wait

```
    fn delay_us<Clock: MicrosClock>(&self, clock: &mut Clock, micros: u32) {
        let start = clock.now_micros();
        while clock.now_micros().wrapping_sub(start) < micros {}
    }
```

- Remember the time, then spin in a loop until the wanted number of microseconds has passed. The `{}` means "do nothing, just wait".

5.7 `wait_for_echo()` — waiting for the echo

```
    fn wait_for_echo<Clock: MicrosClock>(
        &mut self,
        clock: &mut Clock,
        high: bool,
    ) -> Result<u32, SensorError> {
        let start = clock.now_micros();
        loop {
            let level = if high {
                self.echo.is_high()
            } else {
                self.echo.is_low()
            };
            if level.map_err(|_| SensorError::Pin)? {
                return Ok(clock.now_micros());
            }
            if clock.now_micros().wrapping_sub(start) > ECHO_TIMEOUT_US {
                return Err(SensorError::Timeout);
            }
        }
    }
```

- `high: bool` says whether we are waiting for a high or a low signal.
- `loop { ... }` repeats forever until we `return`.
- Each round we read the echo pin's level.
- If it matches what we wait for, hand back the current time.
- If we have waited longer than the timeout, give up with `Timeout`.

6. `src/lib/servo.rs` — the neck

6.1 Friendly angles and hidden numbers

```
use embedded_hal::pwm::SetDutyCycle;

pub const TURN_LEFT_ANGLE: u8 = 45;
pub const TURN_CENTER_ANGLE: u8 = 90;
pub const TURN_RIGHT_ANGLE: u8 = 135;

const MIN_ANGLE: u8 = 0;
const MAX_ANGLE: u8 = 180;
const PERIOD_US: u32 = 20_000;
const MIN_PULSE_US: u32 = 500;
const MAX_PULSE_US: u32 = 2_500;
```

- `SetDutyCycle` is the standard shape of a PWM pin (the servo signal).
- The three `pub const` are the angles we actually use: 45, 90, 135.
- `MIN_ANGLE`/`MAX_ANGLE` (0 and 180) are the safe limits.
- `PERIOD_US` = 20000: the PWM signal repeats every 20000 microseconds (50 times a second).
- `MIN_PULSE_US`/`MAX_PULSE_US` = 500 and 2500: the pulse lengths for the two ends of the servo's travel.

6.2 The head box

```
pub struct Head<Pwm> {
   pwm: Pwm,
   max_duty: u16,
}

impl<Pwm: SetDutyCycle> Head<Pwm> {
   pub fn new(pwm: Pwm) -> Self {
       let max_duty = pwm.max_duty_cycle();
       Self { pwm, max_duty }
   }
```

- `Head<Pwm>` holds the PWM signal and the biggest number it can accept (`max_duty`).
- `new()` asks the PWM pin its maximum (`max_duty_cycle()`), then stores both.

6.3 `set_angle()` — turn to an angle

```
    pub fn set_angle(&mut self, degrees: u8) -> Result<(), Pwm::Error> {
       let degrees = degrees.clamp(MIN_ANGLE, MAX_ANGLE);
       let pulse_us = MIN_PULSE_US
           + (u32::from(degrees) * (MAX_PULSE_US - MIN_PULSE_US)) / u32::from(MAX_ANGLE);
       let duty = (u32::from(self.max_duty) * pulse_us / PERIOD_US) as u16;
       self.pwm.set_duty_cycle(duty)
   }
```

- `clamp(0, 180)` keeps the angle inside the safe range, even if someone asks for something silly.
- The next line turns the angle into a pulse length: at 0 degrees it is 500, at 180 degrees it is 2500.
- Then it turns the pulse into the number the Pico understands (`duty`) and sends it with `set_duty_cycle`.

```
    pub fn center(&mut self) -> Result<(), Pwm::Error> {
       self.set_angle(TURN_CENTER_ANGLE)
   }
}
```

- `center()` is a shortcut: look straight ahead (90 degrees).

7. `src/lib/audio.rs` — the voice

7.1 The track and the promise

```
use embedded_io::Write;

pub const FIRST_TRACK: u16 = 1;

pub trait AudioPlayer {
   type Error;

   fn play_track(&mut self, track: u16) -> Result<(), Self::Error>;
}
```

- `Write` is the standard shape for "something we can send bytes to".
- `FIRST_TRACK` = 1: the scary song is the first track on the SD card.
- `AudioPlayer` is a promise: "I can play a track number".

7.2 The frame's special numbers

```
const FRAME_START: u8 = 0x7E;
const FRAME_END: u8 = 0xEF;
const FRAME_VERSION: u8 = 0xFF;
const FRAME_LENGTH: u8 = 0x06;
const CMD_PLAY_TRACK: u8 = 0x03;
```

- A **frame** is a small message of 10 numbers.
- `0x7E` means "message starts here"; `0xEF` means "message ends here".
- `0x06` says how many numbers follow the length.
- `0x03` is the command "play a track".
- `0x` in front of a number means it is written in hexadecimal (base 16), which chip makers like to use.

7.3 The player box

```
pub struct DfPlayer<Writer> {
   writer: Writer,
}

impl<Writer> DfPlayer<Writer> {
```

```
    pub fn new(writer: Writer) -> Self {
        Self { writer }
    }
}
```

- `DfPlayer<Writer>` holds anything we can write bytes to (our UART).
- `new()` stores it.

```
impl<Writer: Write> AudioPlayer for DfPlayer<Writer> {
    type Error = Writer::Error;

    fn play_track(&mut self, track: u16) -> Result<(), Self::Error> {
        let frame = play_track_frame(track);
        self.writer.write_all(&frame)
    }
}
```

- `impl<Writer: Write> AudioPlayer for DfPlayer<Writer>` means "a `DfPlayer` keeps the `AudioPlayer` promise, as long as its writer can write".
- `play_track()` builds the 10-number frame and sends all of it with `write_all`.

7.4 Building the frame

```
fn play_track_frame(track: u16) -> [u8; 10] {
    let [folder, number] = track.to_be_bytes();
    let checksum = frame_checksum(CMD_PLAY_TRACK, folder, number);
    [
        FRAME_START,
        FRAME_VERSION,
        FRAME_LENGTH,
        CMD_PLAY_TRACK,
        0x00,
        folder,
        number,
        (checksum >> 8) as u8,
        checksum as u8,
        FRAME_END,
    ]
}
```

- `[u8; 10]` means "exactly 10 bytes".
- `to_be_bytes()` splits the track number into two bytes (folder and number).
- `frame_checksum(...)` computes a safety number.

- The list in brackets is the finished message, in order: start, version, length, command, feedback, folder, number, checksum-high, checksum-low, end.
- `checksum >> 8` takes the top half of the checksum; `checksum as u8` takes the bottom half.

7.5 The checksum

```
fn frame_checksum(command: u8, parameter_high: u8, parameter_low: u8) -> u16 {
   let sum = u16::from(FRAME_VERSION)
       + u16::from(FRAME_LENGTH)
       + u16::from(command)
       + u16::from(parameter_high)
       + u16::from(parameter_low);
   0u16.wrapping_sub(sum)
}
```

- Add up the important numbers of the message.
- `0u16.wrapping_sub(sum)` is a maths trick (called "two's complement") that gives a number the player can use to check the message was not scrambled.

7.6 The test

```
#[cfg(test)]
mod tests {
   use super::*;

   #[test]
   fn first_track_frame_matches_the_datasheet_example() {
       assert_eq!(
           play_track_frame(FIRST_TRACK),
           [0x7E, 0xFF, 0x06, 0x03, 0x00, 0x00, 0x01, 0xFE, 0xF7, 0xEF]
       );
   }
}
```

- This test builds the frame for track 1 and checks it matches the exact example from the player's manual. If we ever break the message, the test fails.

8. `src/main.rs` — the body

8.1 Special notes at the top

```
#![no_std]
#![no_main]
```

- `#![no_std]` says "no big standard library" (there is no operating system).
- `#![no_main]` says "I will not use the normal `main`; the chip's startup will call mine".

```
use defmt::info;
use defmt_rtt as _;
use panic_halt as _;

use cortex_m::delay::Delay;
use cortex_m_rt::entry;
use rp_pico as bsp;
```

- `use defmt::info;` brings in the log macro `info!`.
- `use defmt_rtt as _;` and `use panic_halt as _;` turn on two helpers by just importing them (the _ means "we do not need a name").
- `Delay` lets us wait; `entry` marks the program's start; `rp_pico` gives us the Pico's parts, nicknamed `bsp` ("board support package").

```
use bsp::hal::{
   clocks::init_clocks_and_plls,
   fugit::RateExtU32,
   pac,
   pwm::Slices as PwmSlices,
   sio::Sio,
   uart::{DataBits, StopBits, UartConfig, UartPeripheral},
   watchdog::Watchdog,
   Clock, Timer,
};
```

- These bring in the Pico tools we need: clock setup, the timer, the pins controller (`Sio`), the UART (for the MP3), the PWM (for the servo), and the watchdog (a safety timer).

```
use scary_monster_lib::audio::{AudioPlayer, DfPlayer, FIRST_TRACK};
use scary_monster_lib::sensor::{HcSr04, MicrosClock};
use scary_monster_lib::servo::{Head, TURN_CENTER_ANGLE, TURN_LEFT_ANGLE, TURN_RIGHT_ANGLE};
use scary_monster_lib::state_machine::{Direction, Reading, State, StateMachine};
```

- These bring in the **brain** pieces we built earlier. The body uses the brain, never the other way round.

8.2 The real clock for the eyes

```
struct RpTimer(Timer);
```

```
impl MicrosClock for RpTimer {
   fn now_micros(&mut self) -> u32 {
       self.0.get_counter().ticks() as u32
   }
}
```

- `struct RpTimer(Timer);` is a small wrapper around the Pico's timer.
- `impl MicrosClock for RpTimer` keeps the brain's promise: now the sensors can ask "how many microseconds have passed?".
- `get_counter().ticks()` reads the timer, and `as u32` shrinks it to our small number.

8.3 The start of the program

```
#[entry]
fn main() -> ! {
```

- `#[entry]` marks where the chip should start.
- `fn main() -> !` means this function never ends (the `!` means "never returns"). That is normal for firmware: it runs forever.

```
    let mut pac = pac::Peripherals::take().unwrap();
   let core = pac::CorePeripherals::take().unwrap();
   let mut watchdog = Watchdog::new(pac.WATCHDOG);
   let sio = Sio::new(pac.SIO);
```

- `pac::Peripherals::take()` grabs all the chip's parts once. `unwrap()` means "if this fails, stop" (it only fails if we call it twice).
- `CorePeripherals` gives us the chip's core (including the system timer).
- `Watchdog` is a safety timer that reboots the chip if the program freezes.
- `Sio` controls the pins.

```
    let clocks = init_clocks_and_plls(
       bsp::XOSC_CRYSTAL_FREQ,
       pac.XOSC,
       pac.CLOCKS,
       pac.PLL_SYS,
       pac.PLL_USB,
       &mut pac.RESETS,
       &mut watchdog,
   )
   .ok()
   .unwrap();
```

- This sets up the chip's heartbeats (clocks). The Pico normally runs at 125 MHz. `.ok().unwrap()` means "if it worked, keep the result".

```
    let mut delay = Delay::new(core.SYST, clocks.system_clock.freq().to_Hz());
    let pins = bsp::Pins::new(pac.IO_BANK0, pac.PADS_BANK0, sio.gpio_bank0, &mut pac.RESETS);
    let mut clock = RpTimer(Timer::new(pac.TIMER, &mut pac.RESETS, &clocks));
```

- `delay` is a helper to wait (used at the end of the loop).
- `pins` gives us names for every pin on the board (`gpio0`, `gpio1`, ...).
- `clock` wraps the Pico timer so the sensors can measure microseconds.

8.4 Building the eyes

```
    let mut left_sensor = HcSr04::new(
        pins.gpio2.into_push_pull_output(),
        pins.gpio3.into_floating_input(),
    );
    let mut right_sensor = HcSr04::new(
        pins.gpio4.into_push_pull_output(),
        pins.gpio5.into_floating_input(),
    );
```

- The left eye uses GP2 as its trigger (an output pin) and GP3 as its echo (an input pin).
- The right eye uses GP4 and GP5.
- `into_push_pull_output()` and `into_floating_input()` change the pin into the kind we need.

8.5 Building the voice

```
    let uart_pins = (pins.gpio0.into_function(), pins.gpio1.into_function());
    let uart_config = UartConfig::new(9600.Hz(), DataBits::Eight, None, StopBits::One);
    let uart = UartPeripheral::new(pac.UART0, uart_pins, &mut pac.RESETS)
        .enable(uart_config, clocks.peripheral_clock.freq())
        .unwrap();
    let mut audio = DfPlayer::new(uart);
```

- GP0 and GP1 become the two UART wires (send and receive).
- The config says: speed 9600, 8 data bits, no parity, 1 stop bit. These match the MP3 player.
- `UartPeripheral::new(...).enable(...)` switches the UART on.
- `DfPlayer::new(uart)` wraps it as our audio player.

8.6 Building the neck

```
    let pwm_slices = PwmSlices::new(pac.PWM, &mut pac.RESETS);
    let mut pwm = pwm_slices.pwm0;
    pwm.set_div_int(64);
    pwm.set_div_frac(0);
    pwm.set_top(39_062);
    pwm.enable();
    pwm.channel_a.output_to(pins.gpio16);
    let mut head = Head::new(pwm.channel_a);
```

- Grab the PWM block and pick slice `pwm0`.
- The divider (64) slows the counter so the pulse repeats about 50 times a second.
- `set_top(39_062)` sets the counting limit for that speed.
- `enable()` turns the PWM on.
- `output_to(pins.gpio16)` sends the PWM signal out on pin GP16.
- `Head::new(...)` wraps that channel as the head servo.

8.7 The loop

```
    let mut machine = StateMachine::new();
    info!("scary monster ready");

    loop {
```

- `machine` is the brain's mood machine, starting in `Idle`.
- `info!("...")` prints a log message (you see it with `make logs`).
- `loop {` starts the endless repeat.

```
        let reading = Reading {
            left_cm: left_sensor.distance_cm(&mut clock).ok(),
            right_cm: right_sensor.distance_cm(&mut clock).ok(),
        };
```

- Ask the left and right eyes for a distance.
- `.ok()` turns a failure into `None` ("nothing seen"), so the program never crashes; it just behaves as if that eye saw nothing.

```
        let previous = machine.state();
        let state = machine.update(reading);
```

- Remember the previous mood (needed so the song plays only once).
- Ask the brain for the new mood.

```
        match state {
           State::Idle => {
               head.center().ok();
           }
           State::Follow => {
               let angle = match reading.direction() {
                   Direction::Left => TURN_LEFT_ANGLE,
                   Direction::Center => TURN_CENTER_ANGLE,
                   Direction::Right => TURN_RIGHT_ANGLE,
               };
               head.set_angle(angle).ok();
           }
           State::Action => {
               head.set_angle(TURN_CENTER_ANGLE).ok();
               if previous != State::Action {
                   info!("person close: playing track {}", FIRST_TRACK);
                   audio.play_track(FIRST_TRACK).ok();
               }
           }
       }
```

- `match state` chooses what to do for each mood:
- `Idle`: look straight ahead.
- `Follow`: look left, centre, or right, based on `reading.direction()`.
- `Action`: look centre, and **if we were not already in Action**, play the song. The `if previous != State::Action` check is what stops the song from restarting every loop.
- `.ok()` on the head and audio calls means "ignore a small failure and keep going".

```
        delay.delay_ms(50);
    }
}
```

- Wait 50 milliseconds, then loop again. This keeps the loop calm and stops the servo from buzzing.

9. How it all runs together

1. The chip starts at `#[entry] fn main()`.
2. The body sets up clocks, pins, the timer, the two eyes, the voice, and the neck.
3. It creates the brain's `StateMachine`.
4. Forever, it repeats: read the eyes -> ask the brain -> move the head and maybe play the song -> wait 50 ms.

The brain (`src/lib/`) never touches real pins, so we can test it on a computer with `make test`. The body (`src/main.rs`) only connects and repeats. Simple, and easy to change: to move a wire, edit the body; to change a rule, edit the brain.